Database Systems Lab 02

E-Commerce Data Queries

Muhammad Ayyan Khan

BS Data Science - Semester 4

1 Introduction

This document presents the completed Lab 02 exercises for the Database Systems course. The lab focuses on fundamental SQL query operations using an e-commerce database schema. All queries were executed in **DBeaver** (visual interface) and **psql** (command-line), demonstrating proficiency with both professional database tools.

1.1 Database Schema Overview

The e-commerce database consists of four interconnected tables:

- **customers**: Stores customer information (customer_id, name, email, city, signup_date)
- **products**: Contains product catalog (product_id, product_name, category, price, stock_qty)
- **orders**: Records customer orders (order_id, customer_id, order_date, status, total_amount, shipped_date)
- **order_items**: Line items for each order (item_id, order_id, product_id, quantity, unit_price)

1.2 Tools Used

- **DBeaver 25.3.3**: Primary GUI for query execution and result visualization
- **PostgreSQL (psql)**: Command-line interface for database operations
- **Database**: lab1_db hosted on localhost:5432

2 Query Explanations and Results

This section presents all 10 queries with code, screenshots, and detailed explanations.

2.1 Query 1: Explore the Data

Objective: Understand the structure and content of all four tables before writing complex queries.

SQL Code:

```sql
SELECT * FROM customers LIMIT 5;
SELECT * FROM products LIMIT 5;
SELECT * FROM orders LIMIT 5;
SELECT * FROM order_items LIMIT 5;
```

Explanation:

- Uses `SELECT *` to retrieve all columns from each table
- `LIMIT 5` restricts output to first 5 rows for quick inspection
- Helps identify data types, relationships, and potential data quality issues
- Standard practice when working with unfamiliar databases

Key Observations:

- **customers**: Contains customer details with unique email addresses
- **products**: Has Electronics, Books, and Furniture categories
- **orders**: Links to customers via customer_id foreign key
- **order_items**: Many-to-many relationship between orders and products
- **shipped_date**: Some values are NULL (orders not yet shipped)

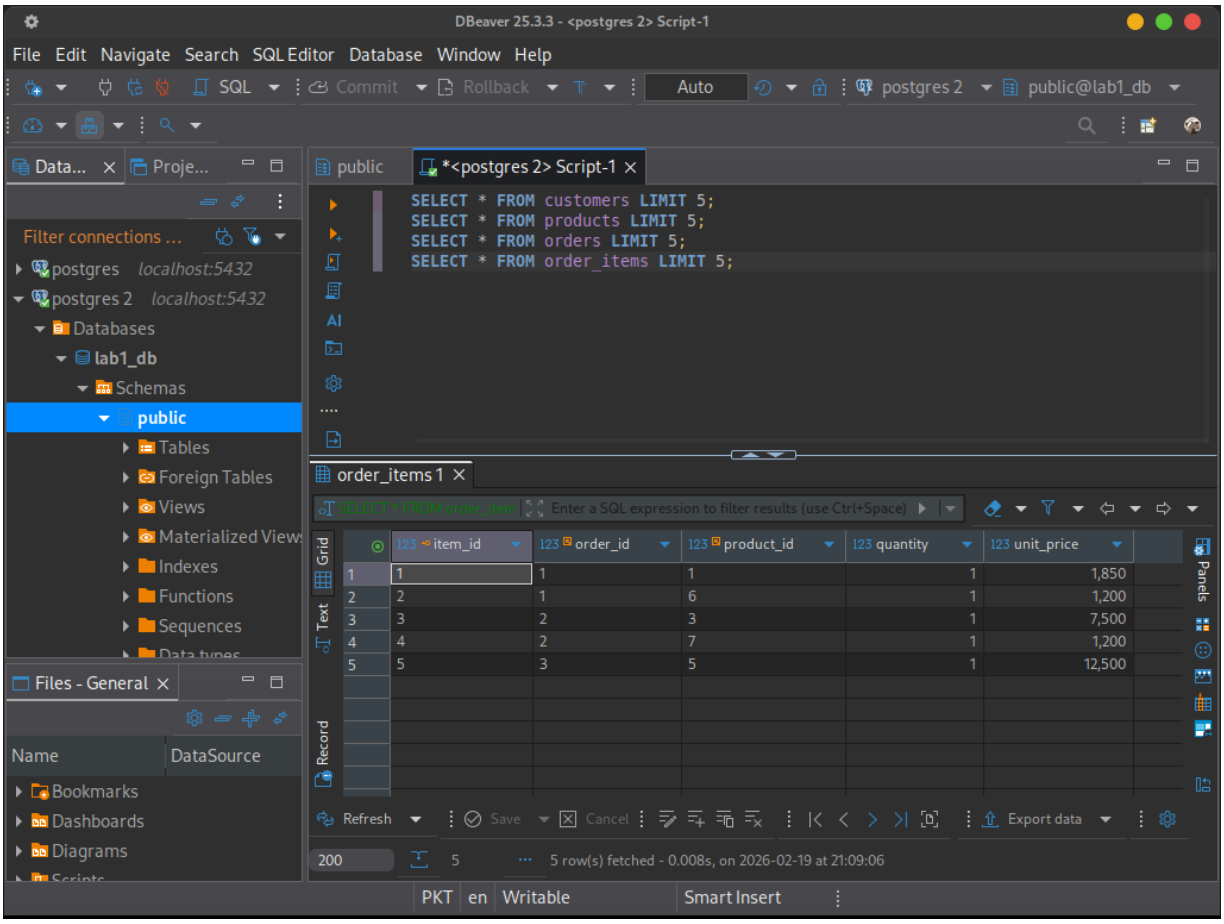

Figure 1: Query 1: Exploring all four tables with LIMIT 5

2.2 Query 2: Select Specific Columns

Objective: Retrieve only customer name, city, and signup date, sorted by registration date.

SQL Code:

```sql
SELECT name, city, signup_date
FROM customers
ORDER BY signup_date;
```

Explanation:

- Selects only 3 specific columns instead of all columns
- `ORDER BY signup_date` sorts results chronologically (oldest first)
- Demonstrates column projection - selecting only needed data
- Results show customer growth over time (2023-2024)

Alternative: Using `ORDER BY signup_date DESC` would show newest customers first.

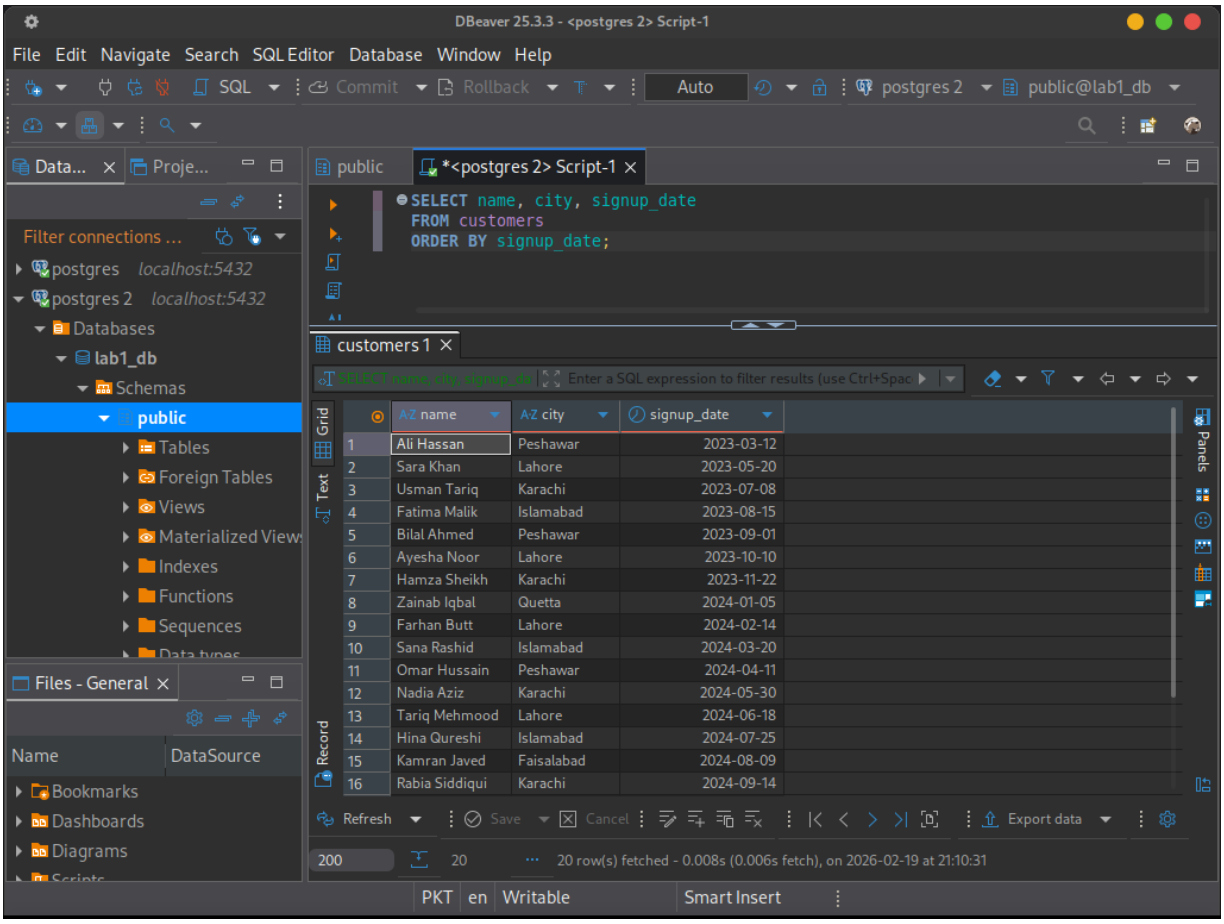

Figure 2: Query 2: Customer names, cities, and signup dates sorted chronologically

2.3 Query 3: Filter by Status

Objective: Find all orders that have been delivered.

SQL Code:

```sql
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE status = 'delivered'
ORDER BY order_date DESC;
```

Explanation:

- `WHERE status = 'delivered'` filters only completed orders
- `ORDER BY order_date DESC` shows most recent deliveries first
- Demonstrates filtering with equality condition
- Useful for fulfillment tracking and delivery analytics

Analysis: By changing the status value to 'pending', 'processing', 'shipped', or 'cancelled', we can analyze order distribution across different statuses.

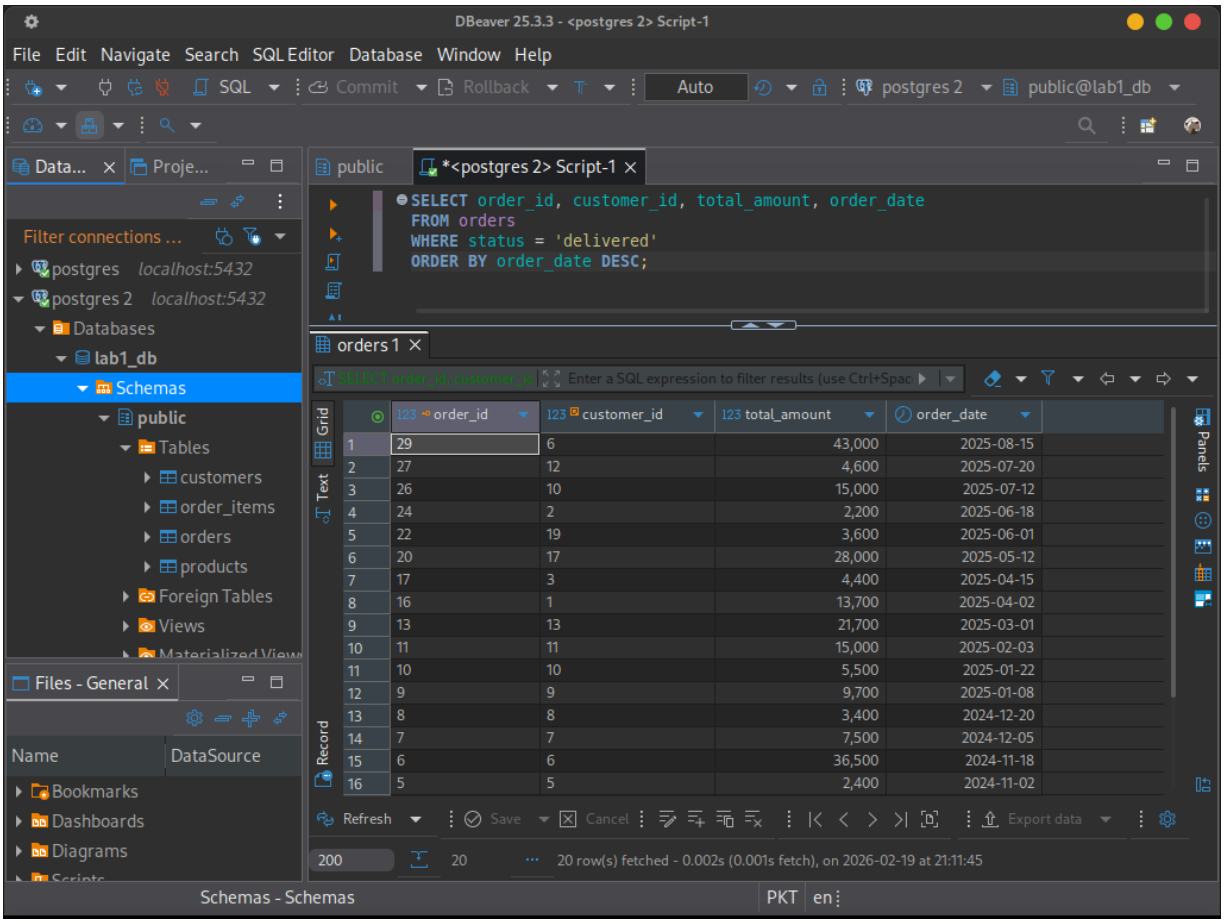

Figure 3: Query 3: All delivered orders sorted by date (newest first)

2.4 Query 4: Filter by Price Range

Objective: Find all products priced between 1,000 and 5,000 rupees using two different methods.

SQL Code:

```sql
-- Method A: BETWEEN operator
SELECT product_name, category, price
FROM products
WHERE price BETWEEN 1000 AND 5000
ORDER BY price;

-- Method B: Comparison operators
SELECT product_name, category, price
FROM products
WHERE price >= 1000 AND price <= 5000
ORDER BY price;
```

Explanation:

- **Method A** uses `BETWEEN` - more concise and readable
- **Method B** uses explicit comparison operators - more flexible

- Both methods are **inclusive** (include boundary values 1000 and 5000)
- Both queries return identical results (9 products)
- Results sorted by price ascending

Key Learning: `BETWEEN X AND Y` is equivalent to `>= X AND <= Y`

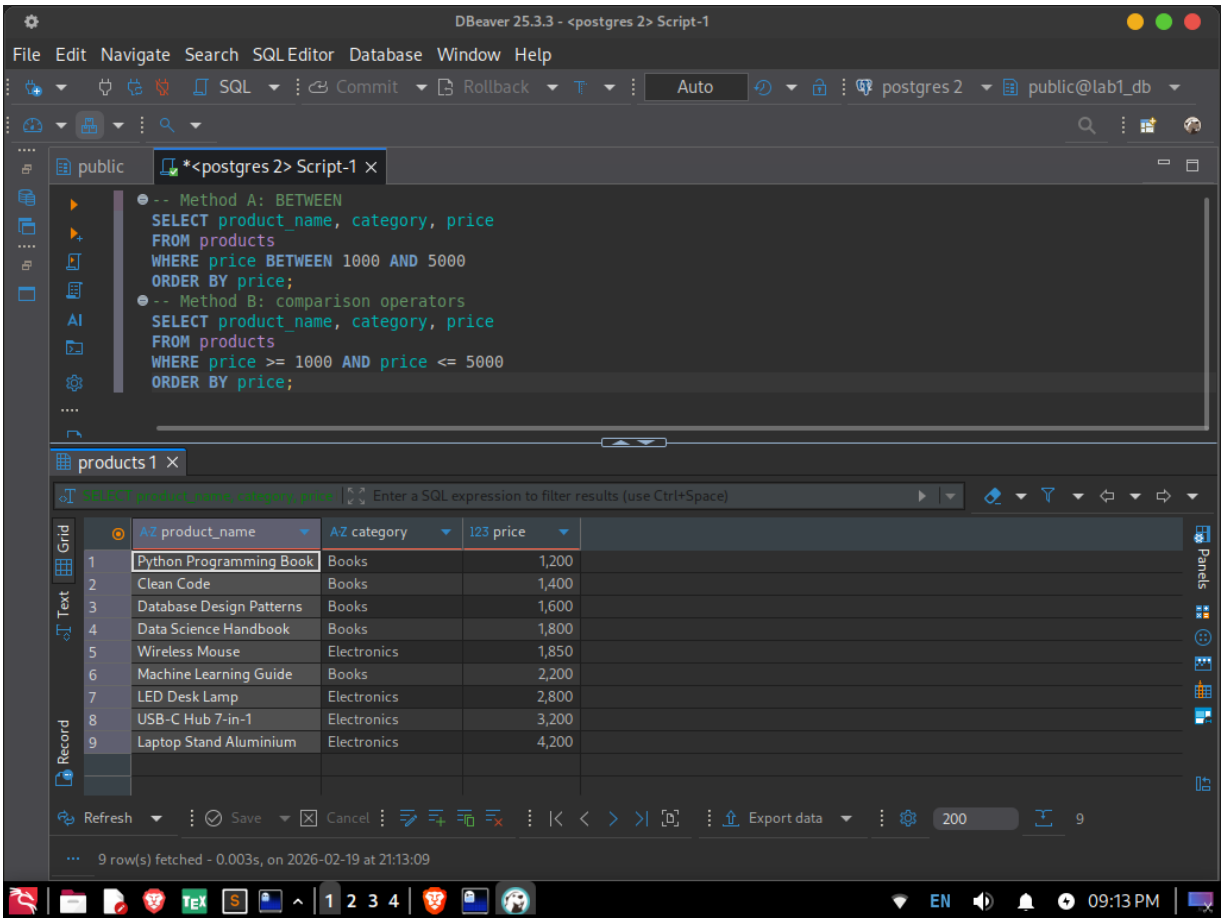

Figure 4: Query 4: Products priced between Rs. 1,000 and Rs. 5,000

2.5 Query 5: Top 10 Highest-Value Orders

Objective: Identify the 10 largest orders by total amount.

SQL Code:

```sql
SELECT order_id, customer_id, total_amount, status
FROM orders
ORDER BY total_amount DESC
LIMIT 10;
```

Explanation:

- `ORDER BY total_amount DESC` sorts highest values first
- `LIMIT 10` restricts to top 10 results

- Useful for identifying VIP customers and high-value transactions
- Important for fraud detection and revenue analysis

Results Analysis:

- Highest order: Rs. 43,000 (order_id 29)
- Top orders span different statuses (delivered, pending)
- Customer 6 appears twice in top 10 (loyal high-value customer)

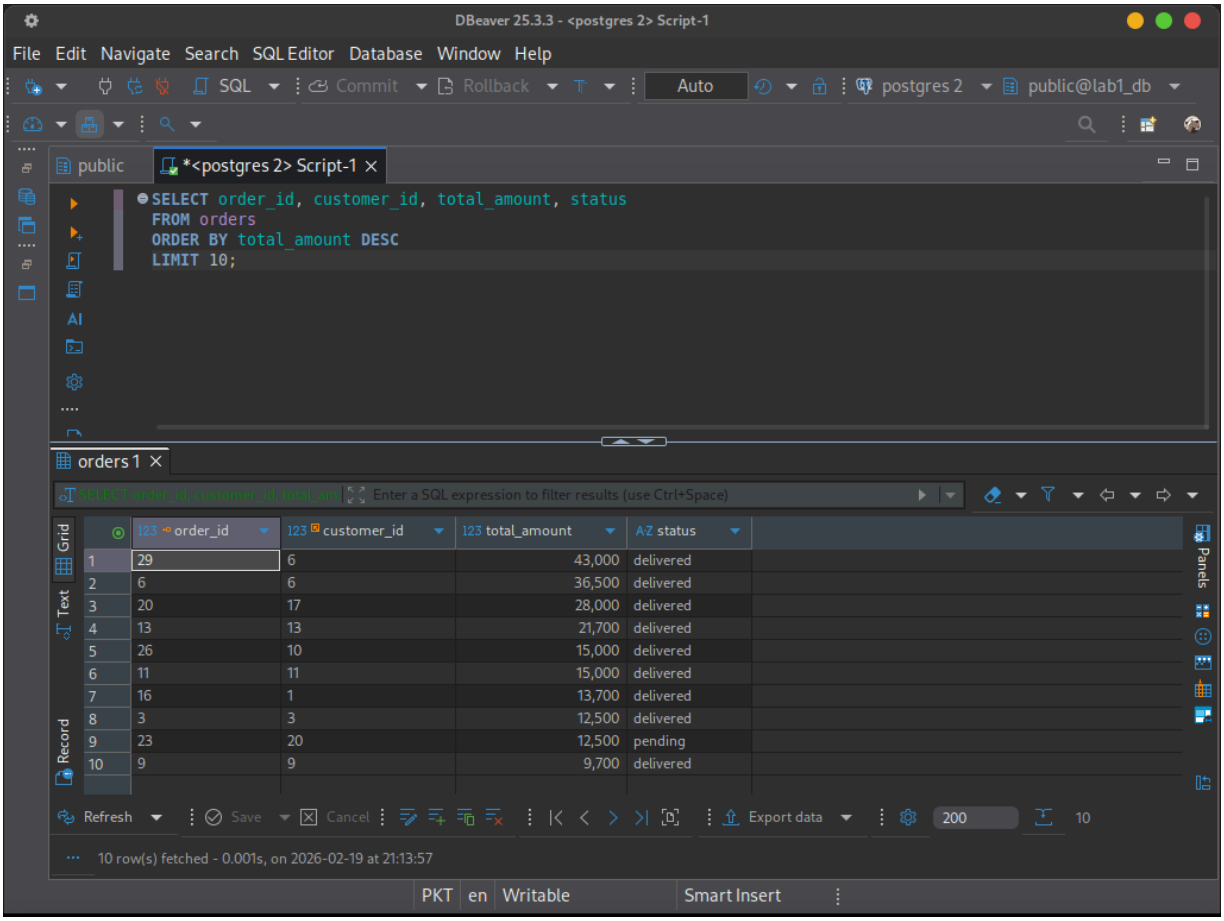

Figure 5: Query 5: Top 10 orders by total amount (descending)

2.6 Query 6: Multi-Condition Filter

Objective: Find delivered orders from 2025 with total above Rs. 10,000.

SQL Code:

```sql
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE status = 'delivered'
  AND order_date >= '2025-01-01'
  AND total_amount > 10000
ORDER BY total_amount DESC;
```

Explanation:

- Combines three conditions with `AND` operator
- Each condition on separate line for readability
- Filters by: status, date range, and monetary threshold
- Results sorted by value (highest first)

Conditions Breakdown:

1. `status = 'delivered'` - Only completed orders
2. `order_date >= '2025-01-01'` - Orders from 2025 onwards
3. `total_amount > 10000` - High-value orders only

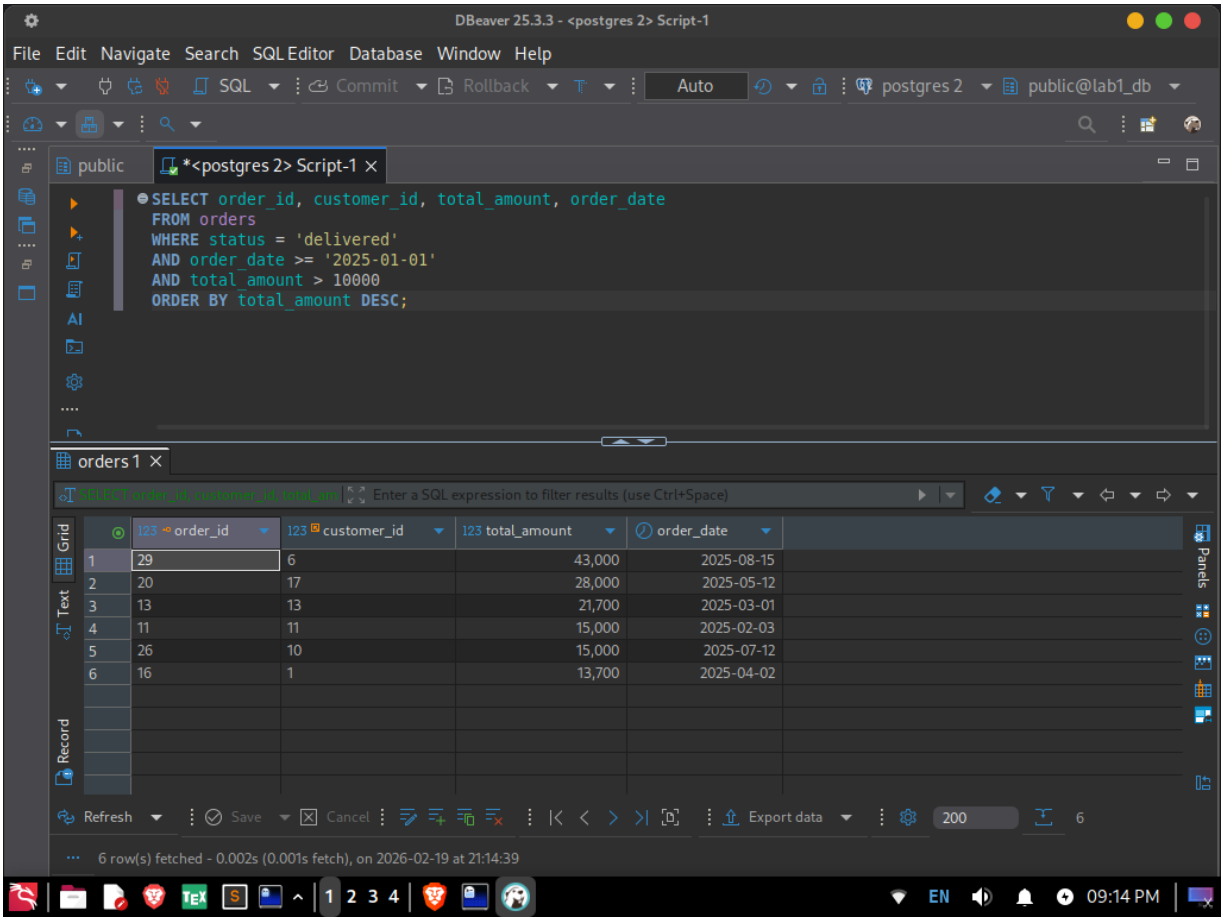

Figure 6: Query 6: Delivered orders from 2025 above Rs. 10,000

2.7 Query 7: Pattern Matching on Email

Objective: Find all customers with Gmail email addresses.

SQL Code:

```sql
SELECT name, email, city
FROM customers
WHERE email LIKE '%@gmail.com'
ORDER BY name;
```

Explanation:

- `LIKE` operator enables pattern matching
- `%@gmail.com` matches any text ending with @gmail.com
- `%` is a wildcard matching zero or more characters
- Results sorted alphabetically by customer name

Pattern Matching Variations:

- `LIKE '%@yahoo%'` - Find Yahoo users
- `LIKE 'a%'` - Names starting with 'a'
- `ILIKE` - Case-insensitive version of LIKE

Results: 12 out of 20 customers (60%) use Gmail addresses.

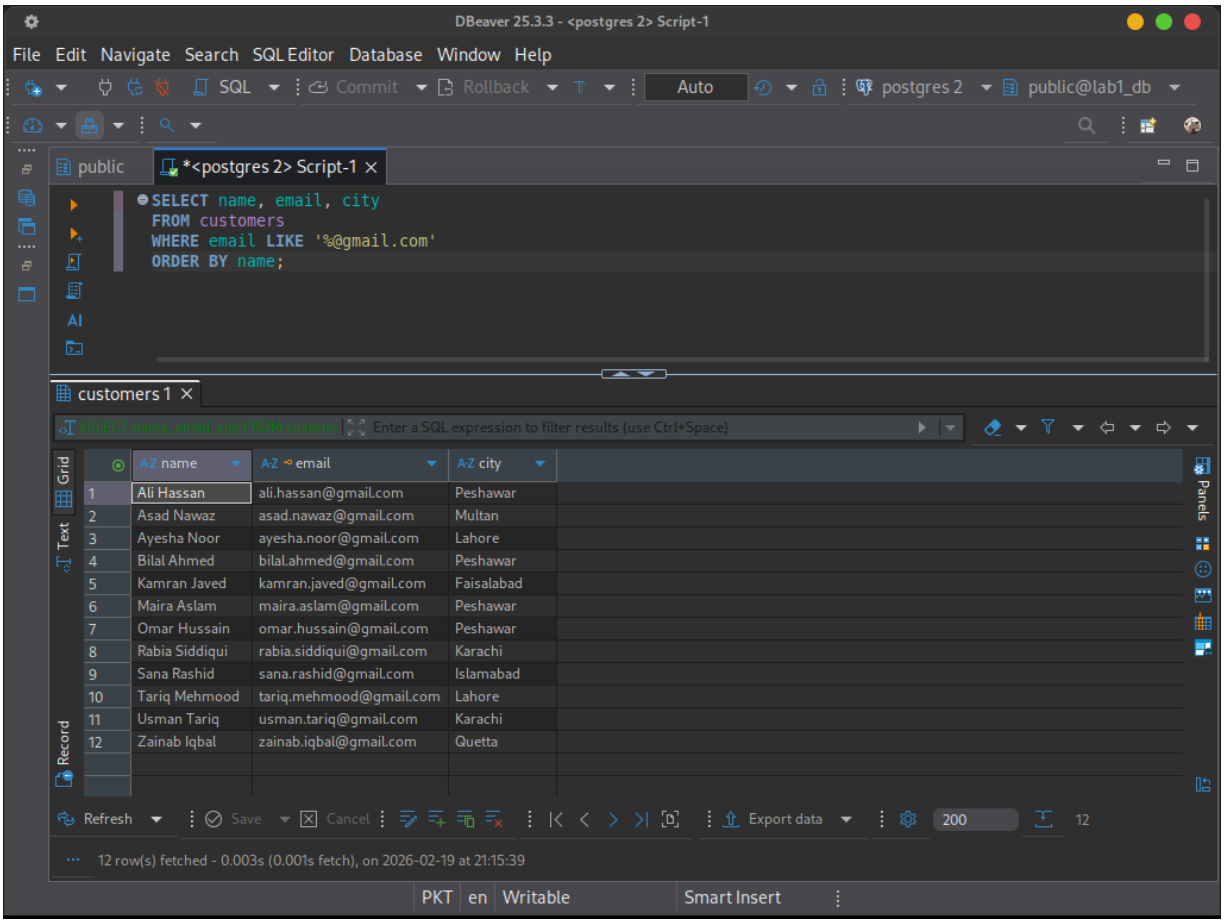

Figure 7: Query 7: Customers with Gmail email addresses

2.8 Query 8: NULL Handling (Unshipped Orders)

Objective: Find all orders that have not yet shipped (shipped_date is NULL).

SQL Code:

```sql
SELECT order_id, customer_id, order_date, status, total_amount
FROM orders
WHERE shipped_date IS NULL
ORDER BY order_date;
```

Explanation:

- `IS NULL` is the ONLY correct way to check for NULL values
- `= NULL` returns zero rows (silently wrong, not an error)
- NULL represents absence of value, not equal to anything
- Identifies orders pending shipment

Important Concept: NULL is not a value - it's the absence of a value. Nothing equals NULL, not even NULL itself.

Results: 7 unshipped orders found with statuses: pending, processing, cancelled.

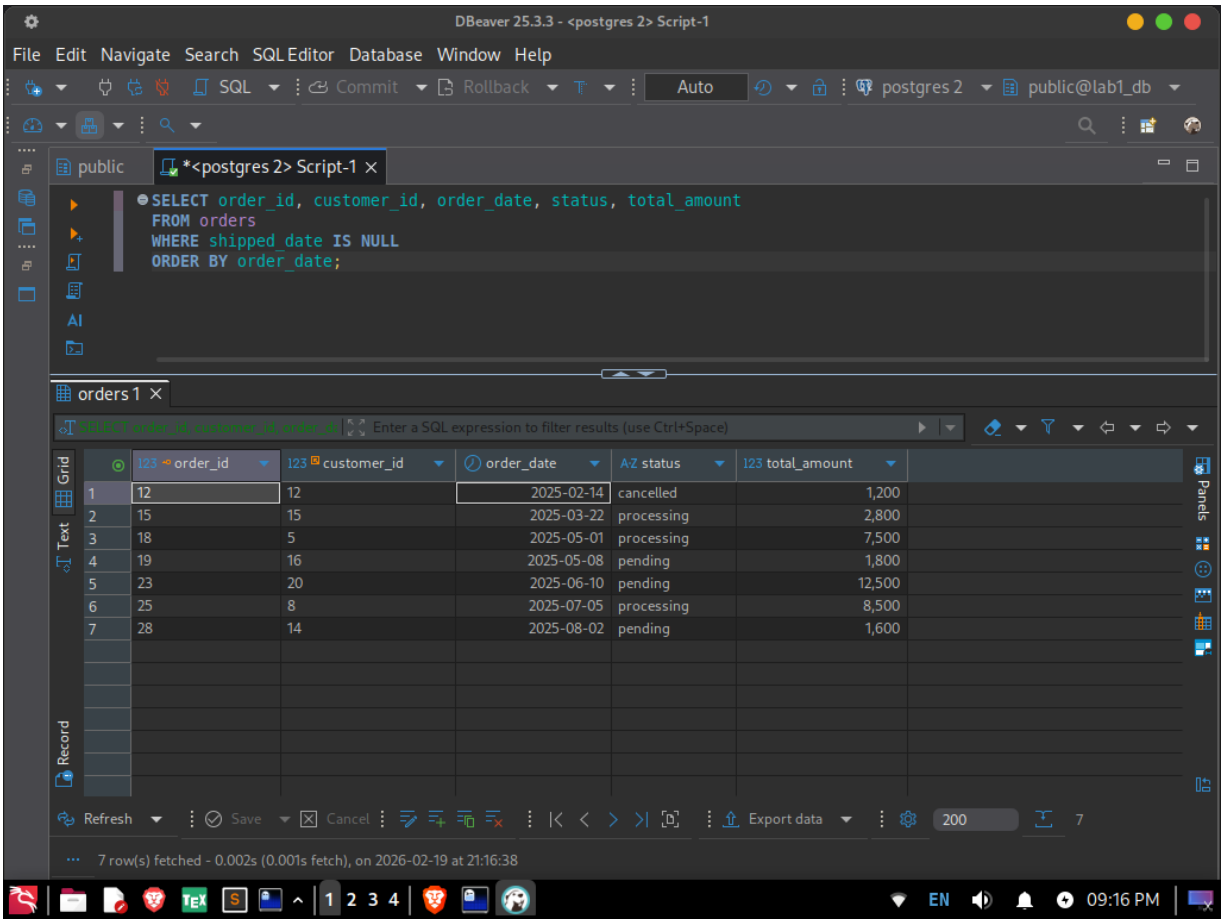

Figure 8: Query 8: Orders with NULL shipped_date (not yet shipped)

2.9 Query 9: Computed Column (Discount Calculation)

Objective: Display products with original price, 20% discounted price, and savings amount.

SQL Code:

```sql
SELECT product_name,
       category,
       price AS original_price,
       ROUND(price * 0.80, 2) AS discounted_price,
       ROUND(price * 0.20, 2) AS you_save
FROM products
ORDER BY discounted_price DESC;
```

Explanation:

- Performs arithmetic directly in SELECT clause
- `price * 0.80` calculates 80% of original (20% discount)
- `price * 0.20` calculates savings amount
- `ROUND(value, 2)` rounds to 2 decimal places
- `AS` creates column aliases for readability

Key Learning: SQL can perform calculations on numeric columns without modifying stored data.

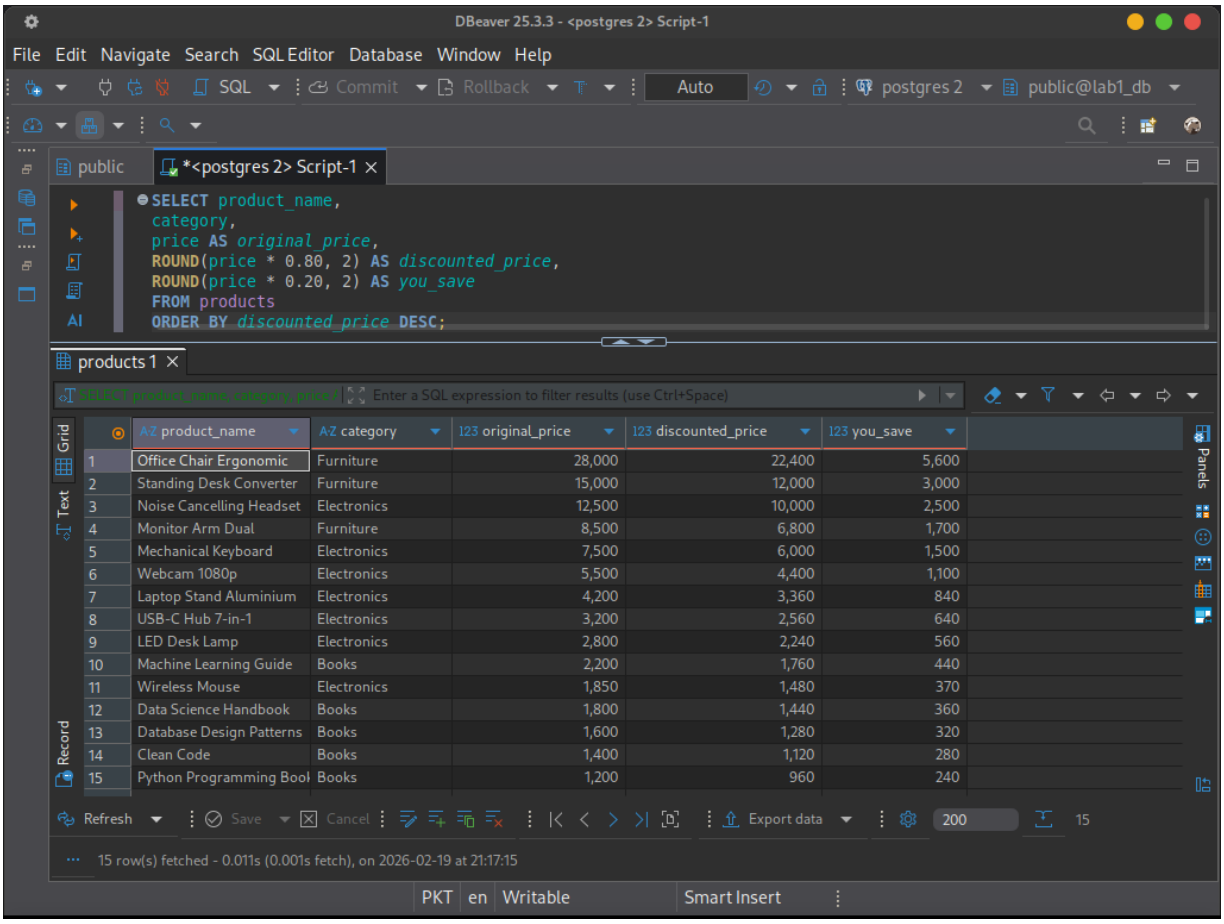

Figure 9: Query 9: Products with 20% discount calculations

2.10 Query 10: Bring Everything Together

Objective: Find top 5 highest-value unshipped orders from 2025 with priority classification.

SQL Code:

```sql
SELECT order_id,
       customer_id,
       total_amount,
       order_date,
       status,
       CASE
           WHEN total_amount > 20000 THEN 'CRITICAL'
           WHEN total_amount > 5000 THEN 'URGENT'
           ELSE 'NORMAL'
       END AS priority
FROM orders
WHERE shipped_date IS NULL
  AND order_date >= '2025-01-01'
ORDER BY total_amount DESC
LIMIT 5;
```

Explanation:

- **CASE WHEN** provides conditional logic (like if/else)
- Three-tier priority system: CRITICAL (¿20k), URGENT (5k-20k), NORMAL (5k)
- Combines NULL handling, date filtering, and sorting
- Real-world application for order fulfillment prioritization

CASE Expression Logic:

1. Check first WHEN condition - if true, return that value
2. If false, check next WHEN condition
3. If no conditions match, use ELSE value

Results: Top unshipped order is Rs. 12,500 (URGENT priority).

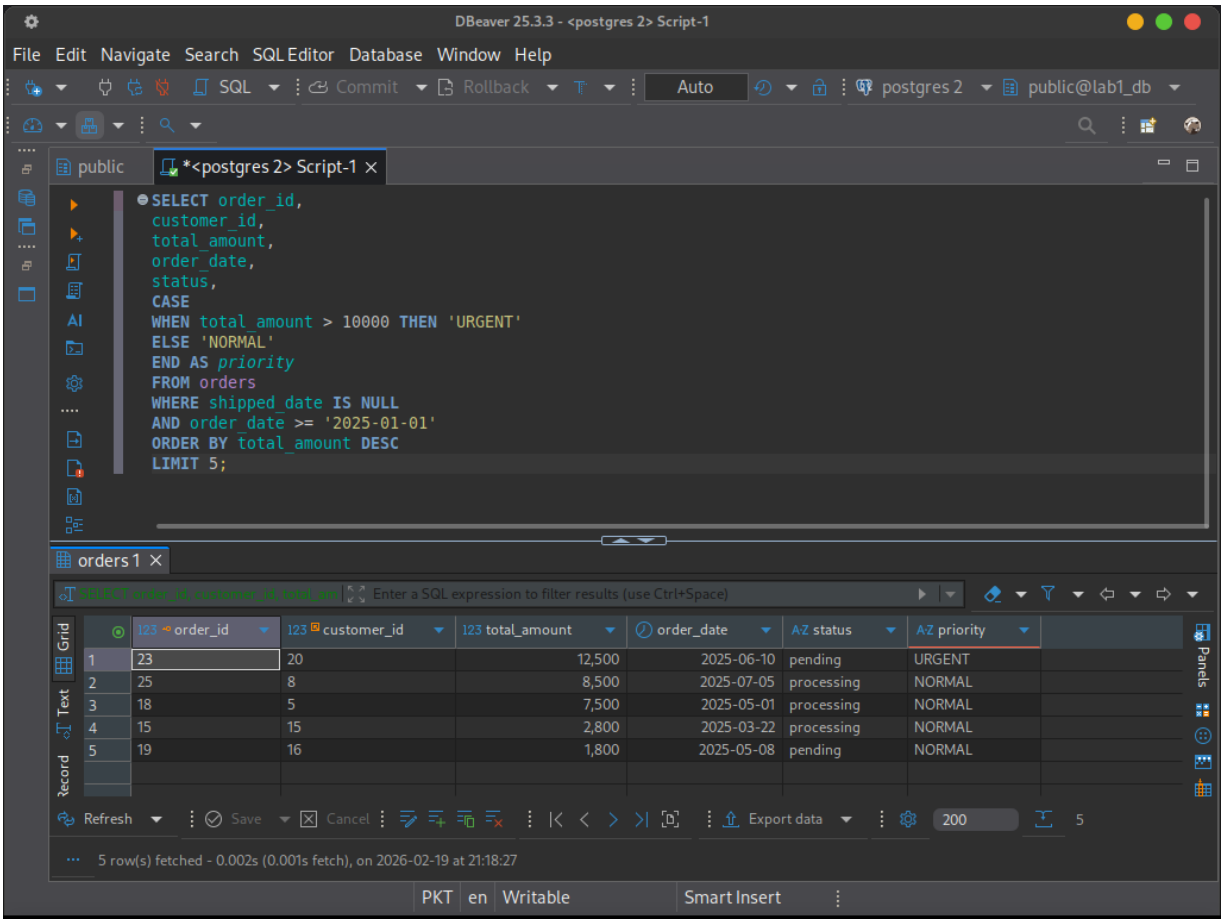

Figure 10: Query 10: Top 5 unshipped 2025 orders with priority classification

3 Summary of SQL Concepts Learned

Table 1: SQL Concepts Covered in Lab 02

Query	Primary Concept	Key Syntax
1	Data Exploration	SELECT *, LIMIT
2	Column Selection	SELECT col1, col2, ORDER BY
3	Filtering	WHERE, equality condition
4	Range Filtering	BETWEEN, comparison operators
5	Sorting and Limiting	ORDER BY DESC, LIMIT
6	Multi-Condition Filter	WHERE with AND
7	Pattern Matching	LIKE, wildcards (%)
8	NULL Handling	IS NULL
9	Computed Columns	Arithmetic, ROUND(), AS
10	Conditional Logic	CASE WHEN, ELSE

4 Reflection

This laboratory exercise reinforced fundamental SQL query writing skills through practical e-commerce scenarios. Key takeaways include:

- **Tool Proficiency:** Gained experience with both DBeaver (GUI) and psql (CLI)
- **NULL Understanding:** Learned that NULL requires special handling with IS NULL
- **Pattern Matching:** Mastered LIKE operator with wildcards for text searches
- **Conditional Logic:** CASE WHEN enables complex business logic in queries
- **Code Readability:** Proper formatting and alignment improves query maintainability

Using both DBeaver and psql demonstrated why professional environments utilize both tools: DBeaver for visual analysis and quick exploration, psql for scripting and automation.

GitHub Repository: `https://github.com/Ayyankhan101/DATABASE-LAB.git`

Files Created:

- `lab2/queries.sql` - All 10 SQL queries
- `lab2/ecommerce_setup.sql` - Database schema and sample data
- `pdf2/lab02_report.tex` - This LaTeX documentation